

Concernant la première partie, j'ai travaillé sur la gestion des données avec PostgreSQL. J'ai créé la table ``patients_hypertension``, importé le fichier CSV dans la base de données, puis effectué plusieurs vérifications : nombre total de lignes, structure des colonnes, types de données, valeurs nulles, doublons, répartition de la variable cible ``Hypertension``, nombre de pays et taux de risque élevé/faible par pays. Le dataset contient 174 982 lignes, avec deux classes pour la variable cible : ``High`` et ``Low``. J'ai aussi constaté que le dataset n'est pas équilibré, car la classe ``High`` est majoritaire par rapport à la classe ``Low``.

Concernant la deuxième partie, j'ai travaillé sur le nettoyage et la préparation des données avec Python. Avant de coder, j'ai pris le temps de regarder plusieurs vidéos théoriques et techniques afin de mieux comprendre les étapes de préparation des données. Pour la partie plus technique, j'ai notamment regardé des vidéos de **Machine Learnia**, ce qui m'a aidée à comprendre la logique générale du prétraitement, de la séparation des données et de l'utilisation de certaines fonctions de scikit-learn.

J'ai ensuite chargé le dataset avec pandas, vérifié les dimensions, les colonnes, les statistiques descriptives, les valeurs manquantes et les doublons. J'ai séparé les variables explicatives ``X`` et la variable cible ``y``, puis j'ai encodé la cible en valeurs numériques avec ``Low = 0`` et ``High = 1``.

J'ai également séparé les variables numériques et les variables catégorielles. Au début, je voulais coder le prétraitement manuellement, sans utiliser directement les fonctions de scikit-learn, afin de mieux comprendre le fonctionnement. J'avais donc commencé à coder manuellement la normalisation des variables numériques et l'encodage des variables catégorielles. Cependant, j'ai utilisé l'IA une première fois pour m'aider à comprendre comment transformer les variables catégorielles en colonnes binaires, car je n'arrivais pas à visualiser comment construire une matrice pour chaque modalité.

Après cela, j'ai utilisé les fonctions adaptées de scikit-learn : ``StandardScaler`` pour les variables numériques, ``OneHotEncoder`` pour les variables catégorielles et ``ColumnTransformer`` pour appliquer le bon traitement aux bonnes colonnes. J'ai gardé mon ancienne méthode manuelle en commentaire dans le code afin de montrer la démarche de compréhension.

J'ai aussi utilisé l'IA une deuxième fois concernant l'ordre du prétraitement et du découpage des données. Au départ, je pensais appliquer le prétraitement sur tout le dataset avant de faire le split. L'IA m'a corrigée en m'expliquant qu'il fallait d'abord séparer les données en train, dev et test, puis apprendre le prétraitement uniquement sur l'ensemble train avec ``fit_transform``, et appliquer les mêmes règles à dev et test avec ``transform``.

Comme le dataset n'est pas équilibré, j'ai aussi utilisé ``stratify`` lors du découpage afin de conserver une répartition similaire des classes ``High`` et ``Low`` dans les ensembles train, dev et test. Actuellement, le dataset est donc séparé en trois ensembles : 70 % pour train, 15 % pour dev et 15 % pour test. Le prétraitement est appliqué correctement aux trois ensembles.

Le code que j'ai actuellement est encore une version de travail. Il n'est pas totalement propre, car j'ai gardé mes commentaires, mes vérifications et mes anciennes tentatives pour suivre mon raisonnement. Je vais ensuite refaire une version plus claire et plus structurée pour le rendu final.

Google doc de la première partie que j'ai partagé avec les autres membres et notre tuteur :

https://docs.google.com/document/d/1drNVvS-lorO9R2dblnEP3ZPx0nM_aHMuJNR4E5S9My4/edit?usp=sharing

Majority Baseline implémenté manuellement

Dans cette partie, j'ai implémenté manuellement un modèle de référence appelé **Majority Baseline**. Ce modèle ne cherche pas à apprendre une relation entre les variables explicatives et la variable cible. Son objectif est simplement de prédire systématiquement la classe la plus fréquente observée dans l'ensemble d'entraînement.

Ce modèle est important car il sert de point de comparaison pour les autres modèles de Machine Learning. En effet, si un modèle plus complexe obtient des performances proches du Majority Baseline, cela signifie qu'il n'apprend pas réellement de structure utile dans les données.

Préparation des données

Comme pour les autres modèles, les données ont d'abord été chargées avec `pandas` à partir du fichier `hypertension_dataset.csv`. Plusieurs vérifications ont été réalisées : dimensions du dataset, affichage des colonnes, statistiques descriptives, valeurs manquantes et doublons.

La variable cible `Hypertension` a ensuite été séparée des variables explicatives. Les modalités de la variable cible ont été encodées de la manière suivante :

Low → 0
High → 1

Les variables explicatives ont été divisées en deux groupes : les variables numériques et les variables catégorielles. Les variables numériques ont été normalisées avec `StandardScaler`, tandis que les variables catégorielles ont été transformées avec `OneHotEncoder`.

Les données ont ensuite été séparées en trois ensembles : train, dev et test. L'ensemble d'entraînement représente 70 % des données, tandis que les ensembles de validation et de test représentent chacun 15 %. Le paramètre `stratify` a été utilisé afin de conserver une répartition similaire des classes dans les trois ensembles. Cette étape est importante car le dataset est déséquilibré, avec une majorité d'individus appartenant à la classe `High`.

Le prétraitement a été appris uniquement sur l'ensemble d'entraînement avec `fit_transform`, puis appliqué aux ensembles de validation et de test avec `transform`. Cela permet d'éviter une fuite de données, car les ensembles de validation et de test ne doivent pas influencer l'apprentissage du prétraitement.

Principe du Majority Baseline

Le Majority Baseline repose sur une logique très simple : il identifie la classe majoritaire dans l'ensemble d'entraînement, puis prédit cette classe pour toutes les observations.

Dans notre implémentation, cette logique est réalisée dans une classe appelée `MajorityBaselineMaison`. Cette classe contient deux méthodes principales : `fit` et `predict`.

La méthode `fit` reçoit uniquement la variable cible `y_train`. Contrairement aux autres modèles, elle n'utilise pas les variables explicatives `X`, car le modèle n'apprend aucune relation entre les caractéristiques des individus et la classe cible. Elle utilise la fonction `np.unique` avec `return_counts=True` afin de compter le nombre d'occurrences de chaque classe. Ensuite, `np.argmax` permet d'identifier l'indice de la classe ayant l'effectif le plus élevé. Cette classe est sauvegardée comme classe majoritaire.

Dans notre cas, la classe majoritaire est la classe `1`, correspondant à `High`. Le modèle apprend donc que la prédiction la plus fréquente dans le train est `High`.

La méthode `predict` reçoit un ensemble de données `X`, mais elle n'utilise pas réellement les valeurs des variables explicatives. Elle utilise seulement le nombre de lignes de `X`, puis crée un tableau de prédictions rempli avec la classe majoritaire apprise pendant l'entraînement. Ainsi, pour chaque individu de l'ensemble de validation, le modèle prédit la même classe.

Évaluation du modèle

Le modèle a été entraîné sur l'ensemble d'entraînement, puis évalué sur l'ensemble de validation. Les prédictions ont été comparées aux vraies valeurs de `y_dev`.

Les métriques utilisées sont l'accuracy, la précision, le rappel, le F1-score et la matrice de confusion. Les métriques ont été calculées séparément pour la classe `High` et pour la classe `Low`, afin de mieux comprendre le comportement du modèle sur chaque classe.

Dans le cas du Majority Baseline, le modèle prédit uniquement la classe `High`. Cela entraîne une accuracy élevée, car la classe `High` est majoritaire dans le dataset. Cependant, cela signifie aussi que le modèle ne reconnaît aucun individu de la classe `Low`.

Les résultats obtenus sont les suivants :

```
Accuracy : 0,719
Recall Low : 0
F1-score Low : 0
```

La matrice de confusion obtenue est de la forme suivante :

```
[[ 0 7380]
 [ 0 18867]]
```

Cette matrice montre que tous les individus de la classe `Low` sont prédits à tort comme `High`. En revanche, tous les individus réellement `High` sont correctement prédits, car le modèle prédit toujours `High`.

Interprétation des résultats

Le Majority Baseline obtient une accuracy relativement élevée, environ 71,9 %. Cependant, cette performance est trompeuse, car elle est principalement due au déséquilibre du dataset. Comme la classe `High` représente la majorité des observations, un modèle qui prédit toujours `High` obtient automatiquement une accuracy proche de la proportion de cette classe.

Ce résultat montre donc que l'accuracy seule n'est pas suffisante pour évaluer les modèles dans ce projet. Il est nécessaire d'analyser aussi la matrice de confusion, le rappel et le F1-score de chaque classe. En

particulier, le rappel et le F1-score de la classe **Low** sont nuls, ce qui montre que le modèle ne parvient pas du tout à identifier cette classe.

Le Majority Baseline n'est donc pas un modèle performant pour la prédiction réelle, mais il est utile comme modèle de référence. Il permet de vérifier si les modèles plus complexes, comme l'arbre de décision manuel ou l'arbre de décision de scikit-learn, apprennent réellement davantage qu'une simple prédiction de la classe majoritaire.

Logique de l'arbre de décision implémenté manuellement

Dans cette partie, j'ai implémenté manuellement un modèle d'arbre de décision afin de mieux comprendre sa logique interne. Contrairement à l'utilisation directe d'une bibliothèque comme scikit-learn, cette implémentation permet de détailler les étapes principales de construction de l'arbre : le calcul de l'impureté, la recherche du meilleur découpage, la construction récursive de l'arbre et la prédiction.

Le principe général d'un arbre de décision est de séparer progressivement les données en plusieurs groupes à l'aide de conditions simples. À chaque nœud de l'arbre, le modèle pose une question de la forme :

`variable <= seuil ?`

Par exemple, après prétraitement, une séparation peut être effectuée sur une variable numérique normalisée ou sur une variable catégorielle encodée en 0 et 1. Les individus qui vérifient la condition sont envoyés dans la branche gauche, tandis que les autres sont envoyés dans la branche droite. Ce processus est répété jusqu'à atteindre une condition d'arrêt. À la fin, chaque feuille de l'arbre correspond à une classe prédite, ici 0 pour **Low** ou 1 pour **High**.

Calcul de l'impureté avec l'indice de Gini

Pour choisir les meilleures séparations, l'arbre utilise l'indice de Gini. Cet indice mesure le degré de mélange des classes dans un groupe. Un groupe est considéré comme pur lorsqu'il contient uniquement une seule classe. Dans ce cas, l'indice de Gini est égal à 0. À l'inverse, si les classes sont fortement mélangées, l'impureté est plus élevée.

Dans notre implémentation, la fonction `gini` commence par compter les classes présentes dans le groupe à l'aide de `np.unique`. Elle calcule ensuite la proportion de chaque classe, puis applique la formule suivante :

$$Gini = 1 - \sum_j p_j^2$$

Par exemple, si un groupe contient uniquement des individus **High**, alors la proportion de la classe **High** est égale à 1 et le Gini vaut 0. Le groupe est donc parfaitement pur. Si un groupe contient à la fois des **High** et des **Low**, le Gini augmente, ce qui signifie que le groupe est moins bien séparé.

L'objectif de l'arbre est donc de trouver des séparations qui diminuent le plus possible l'impureté des groupes obtenus.

Séparation des données en deux branches

La méthode `split` permet de couper les données selon une colonne et un seuil. Pour une colonne donnée, l'algorithme compare chaque valeur au seuil choisi. Les lignes dont la valeur est inférieure ou égale au seuil sont placées dans la partie gauche, tandis que les lignes dont la valeur est strictement supérieure au seuil sont placées dans la partie droite.

La séparation est donc toujours réalisée selon la règle suivante :

```
si X[:, colonne] <= seuil → branche gauche  
sinon → branche droite
```

Cette fonction retourne quatre éléments : les variables explicatives de la branche gauche, les classes associées à la branche gauche, les variables explicatives de la branche droite et les classes associées à la branche droite. Cette étape est essentielle car elle permet ensuite de calculer l'impureté des deux groupes obtenus.

Recherche du meilleur découpage

La méthode `meilleur_split` est l'étape centrale de l'arbre. Elle cherche la meilleure séparation possible parmi plusieurs colonnes et plusieurs seuils. Pour cela, l'algorithme parcourt toutes les colonnes du tableau `X`. Pour chaque colonne, il récupère les valeurs différentes présentes dans cette colonne.

Lorsque la colonne contient peu de valeurs différentes, tous les seuils possibles sont testés. C'est par exemple le cas des variables catégorielles après encodage one-hot, qui prennent généralement les valeurs 0 ou 1. En revanche, lorsque la colonne contient beaucoup de valeurs différentes, tester toutes les valeurs serait trop long. Pour limiter le temps de calcul, seules certaines valeurs représentatives sont testées, correspondant aux percentiles 10, 25, 50, 75 et 90.

Pour chaque seuil testé, les données sont séparées en deux groupes grâce à la fonction `split`. Si l'une des deux branches est vide, la séparation est ignorée, car elle n'apporte aucune information. Sinon, l'algorithme calcule le Gini de la branche gauche et le Gini de la branche droite.

Ensuite, un Gini total est calculé sous forme de moyenne pondérée :

$$\text{Gini total} = \text{poids gauche} \times \text{Gini gauche} + \text{poids droite} \times \text{Gini droite}$$

$$J(k) = \frac{m_{\text{gauche}}}{m} G_{\text{gauche}} + \frac{m_{\text{droite}}}{m} G_{\text{droite}}$$

Les poids correspondent à la proportion d'individus dans chaque branche. Cette pondération est importante, car un groupe contenant beaucoup d'individus doit avoir plus d'influence qu'un petit groupe.

L'algorithme garde ensuite la colonne et le seuil qui donnent le Gini total le plus faible. Cela signifie que la meilleure séparation est celle qui permet d'obtenir les groupes les plus purs possible.

Construction récursive de l'arbre

La construction de l'arbre est réalisée avec la méthode `construire_arbre`. Cette méthode est récursive, c'est-à-dire qu'elle s'appelle elle-même pour construire les branches gauche et droite.

À chaque étape, l'algorithme vérifie d'abord plusieurs conditions d'arrêt. La première condition concerne la profondeur maximale de l'arbre. Si la profondeur actuelle atteint `max_depth`, l'algorithme arrête de diviser le groupe et retourne la classe majoritaire. Cette condition permet d'éviter que l'arbre devienne trop complexe.

La deuxième condition d'arrêt concerne le nombre minimal d'échantillons. Si un groupe contient moins d'individus que `min_samples_split`, l'arbre ne le divise plus et retourne également la classe majoritaire. Cela permet d'éviter des séparations sur des groupes trop petits, qui pourraient conduire à du surapprentissage.

La troisième condition d'arrêt concerne la pureté du groupe. Si tous les individus d'un groupe appartiennent à la même classe, il n'est plus nécessaire de continuer la séparation. Dans ce cas, la classe unique du groupe est retournée directement.

Si aucune condition d'arrêt n'est atteinte, l'algorithme cherche la meilleure séparation à l'aide de la méthode `meilleur_split`. Il sépare ensuite les données en deux branches, puis recommence le même processus sur la branche gauche et sur la branche droite.

Chaque nœud de l'arbre est représenté par un dictionnaire contenant quatre informations :

`colonne` : la variable utilisée pour la séparation
`seuil` : la valeur du seuil
`gauche` : la branche gauche
`droite` : la branche droite

Les branches gauche et droite peuvent elles-mêmes être d'autres nœuds, ou bien des feuilles contenant directement une classe prédite. L'arbre final est donc une structure de dictionnaires imbriqués.

Prédiction avec l'arbre

Une fois l'arbre construit, la prédiction se fait en parcourant l'arbre depuis la racine jusqu'à une feuille. Pour chaque individu, la méthode `predire_ligne` regarde la colonne et le seuil du nœud courant. Si la valeur de l'individu est inférieure ou égale au seuil, l'algorithme suit la branche gauche. Sinon, il suit la branche droite.

Ce processus continue jusqu'à atteindre une feuille. Une feuille n'est plus un dictionnaire, mais directement une classe, soit 0 soit 1. Cette classe est alors retournée comme prédiction finale pour l'individu.

La méthode `predict` applique cette logique à toutes les lignes de l'ensemble de validation. Elle retourne donc un tableau contenant les prédictions du modèle pour tous les individus.

Choix des hyperparamètres

Deux hyperparamètres principaux ont été utilisés dans l'arbre de décision manuel : `max_depth` et `min_samples_split`.

Le paramètre `max_depth` limite la profondeur maximale de l'arbre. Il contrôle donc le nombre maximal de niveaux que l'arbre peut construire. Lorsque la profondeur est faible, l'arbre reste simple et possède peu de règles de décision. Cela limite le risque de surapprentissage, mais peut aussi empêcher le modèle d'apprendre suffisamment les différences entre les classes. Dans notre cas, avec une profondeur faible comme `max_depth = 5`, le modèle prédit presque toujours la classe majoritaire `High`. L'accuracy est alors élevée, environ 0,719, mais le rappel de la classe `Low` est quasiment nul. Cela montre que le modèle est trop simple et qu'il ne parvient pas à reconnaître correctement la classe minoritaire.

À l'inverse, lorsque `max_depth` augmente, l'arbre peut construire davantage de niveaux et donc apprendre des règles plus détaillées. Cela permet au modèle de mieux reconnaître certains individus de la classe `Low`. Par exemple, avec `max_depth = 20` et `min_samples_split = 25`, le rappel de la classe `Low` augmente à environ 0,176, alors qu'il était presque nul avec une profondeur plus faible. Cependant, cette amélioration se fait au prix d'une diminution du rappel de la classe `High`, car le modèle prédit davantage de vrais `High` comme `Low`.

Le paramètre `min_samples_split` indique le nombre minimal d'individus nécessaires dans un nœud pour autoriser une nouvelle séparation. Une valeur faible, comme 15, permet à l'arbre de continuer à diviser des groupes relativement petits. Cela augmente la complexité du modèle et peut l'aider à détecter davantage la classe minoritaire. Cependant, cela peut également conduire à des règles trop spécifiques et augmenter le risque de surapprentissage. Une valeur plus élevée limite les séparations, ce qui rend l'arbre plus stable mais parfois trop général.

Plusieurs combinaisons d'hyperparamètres ont été testées afin d'observer leur influence sur les performances du modèle. Les résultats montrent une évolution progressive du comportement de l'arbre. Avec des paramètres peu profonds, comme `max_depth = 5` et `min_samples_split = 15`, le modèle obtient une accuracy élevée, mais il prédit presque uniquement la classe `High`. Dans ce cas, seulement 3 individus `Low` sont correctement identifiés sur l'ensemble de validation, ce qui montre que le modèle se comporte presque comme un Majority Baseline.

Avec `max_depth = 10` et `min_samples_split = 50`, le modèle reste encore fortement orienté vers la classe majoritaire. Il améliore légèrement la détection de la classe `Low`, avec 110 individus `Low` correctement prédits, mais ce résultat reste insuffisant. L'accuracy reste élevée, environ 0,712, principalement parce que la classe `High` est majoritaire dans le dataset.

La configuration `max_depth = 20` et `min_samples_split = 25` donne un meilleur compromis. Le modèle reconnaît 1 300 individus `Low` correctement, tout en gardant un rappel relativement élevé pour la classe `High`, environ 0,831. L'accuracy diminue à environ 0,647, mais cette baisse est liée au fait que le modèle ne prédit plus uniquement la classe majoritaire. Cette configuration permet donc d'obtenir un modèle plus informatif que les modèles peu profonds.

Des profondeurs plus élevées ont ensuite été testées, notamment `max_depth = 40, 50, 80, 100, 150` et `200`, avec `min_samples_split = 15`. Avec `max_depth = 40`, le modèle reconnaît davantage la classe `Low`, avec 2 262 individus `Low` correctement prédits. Cependant, il augmente fortement le nombre de vrais `High` prédits à tort comme `Low`, avec 5 913 cas. Cela entraîne une baisse importante du rappel de la classe `High`, qui passe à environ 0,687. Dans un contexte médical, ce type d'erreur serait particulièrement problématique, car il reviendrait à classer un individu réellement à risque comme non à risque.

À partir de `max_depth = 40`, les résultats se stabilisent presque totalement. Les configurations `max_depth = 50, 80, 100, 150` et `200` donnent pratiquement les mêmes résultats que `max_depth = 40`. Cela signifie que l'arbre atteint déjà sa limite de séparation avec une profondeur d'environ 40 pour les données et les règles de split utilisées. Augmenter davantage la profondeur ne permet donc plus d'améliorer les performances du modèle, mais augmente seulement le risque de complexité inutile.

Ainsi, l'analyse des hyperparamètres montre que l'augmentation de la profondeur améliore progressivement la reconnaissance de la classe `Low`, mais dégrade en parallèle la capacité du modèle à bien reconnaître la classe `High`. La configuration `max_depth = 20` et `min_samples_split = 25` a donc été retenue comme

meilleur compromis parmi les essais réalisés. Elle permet de dépasser le comportement du Majority Baseline, tout en évitant de trop augmenter le nombre de vrais **High** prédits à tort comme **Low**.

Évaluation du modèle

Le modèle a été entraîné sur l'ensemble d'entraînement puis évalué sur l'ensemble de validation. Les métriques utilisées sont l'accuracy, la précision, le rappel, le F1-score et la matrice de confusion. Ces métriques permettent d'analyser les performances globales du modèle, mais aussi son comportement sur chaque classe.

Les résultats obtenus avec l'arbre de décision manuel sont les suivants :

```
Accuracy : 0.6471596753914733
Precision High : 0.7206652577414316
Recall High : 0.8313987385381884
F1 High : 0.7720818054290848
Precision Low : 0.2901138138808302
Recall Low : 0.17615176151761516
F1 Low : 0.2192058005227215
```

La matrice de confusion obtenue est :

```
[[ 1300  6080]
 [ 3181 15686]]
```

Cette matrice montre que le modèle reconnaît correctement une grande partie des individus **High**, avec 15 686 vrais **High** bien prédits. Cependant, 3 181 vrais **High** sont prédits à tort comme **Low**. Concernant la classe **Low**, 1 300 individus sont correctement reconnus, mais 6 080 sont encore prédits comme **High**.

Ces résultats montrent que l'arbre manuel apprend davantage qu'un simple modèle de référence qui prédit toujours la classe majoritaire. Cependant, la classe **Low** reste difficile à prédire. Cela peut s'expliquer par le déséquilibre du dataset, mais aussi par le caractère simplifié de l'implémentation manuelle. Contrairement à scikit-learn, cette version ne contient pas toutes les optimisations possibles et ne teste qu'un nombre limité de seuils pour réduire le temps de calcul.

L'intérêt principal de cette implémentation est donc pédagogique : elle permet de comprendre comment un arbre de décision choisit ses séparations, comment il construit ses branches et comment il effectue ses prédictions.

Démarche de compréhension et aides utilisées

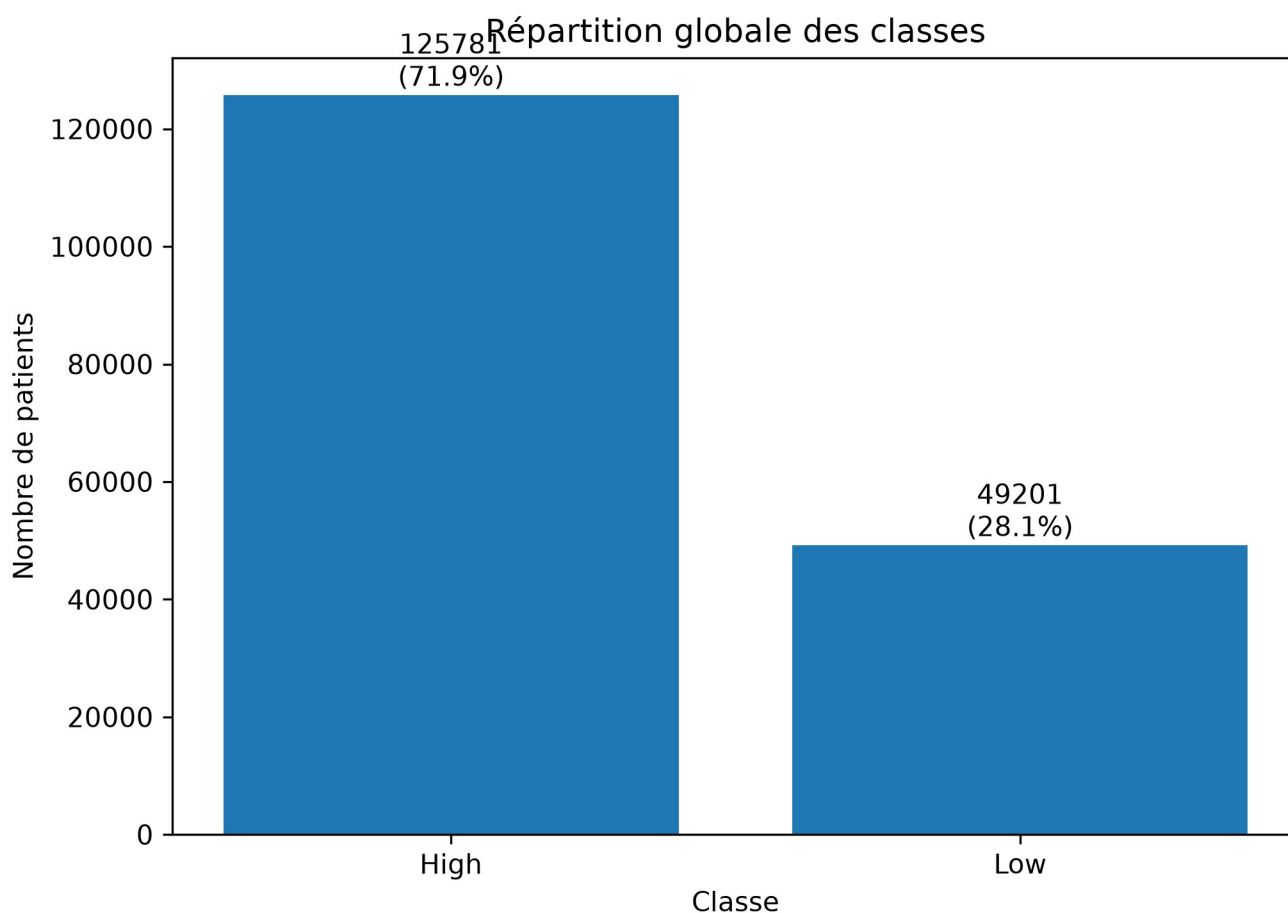
Pour mieux comprendre la logique des arbres de décision, j'ai consulté plusieurs ressources pédagogiques en ligne, notamment des vidéos YouTube comme celles de la chaîne **AI For You**. Ces vidéos m'ont aidée à comprendre le principe général d'un arbre de décision : la séparation progressive des données, le calcul de l'impureté, le choix du meilleur seuil et l'influence des hyperparamètres comme `max_depth` et `min_samples_split`.

J'ai également utilisé des outils d'intelligence artificielle comme aide ponctuelle afin de clarifier certaines parties techniques de l'implémentation. Cette aide m'a notamment permis de mieux comprendre la structuration de la classe `ArbreDecisionMaison`, en particulier la fonction `meilleur_split`. Cette fonction est une étape centrale du modèle, car elle teste plusieurs colonnes et plusieurs seuils afin d'identifier la séparation qui minimise le mieux l'impureté de Gini. L'intelligence artificielle a donc été utilisée comme support de compréhension et non comme remplacement du travail d'implémentation, afin

de mieux assimiler la logique de parcours des colonnes, de test des seuils et de comparaison des impuretés obtenues.

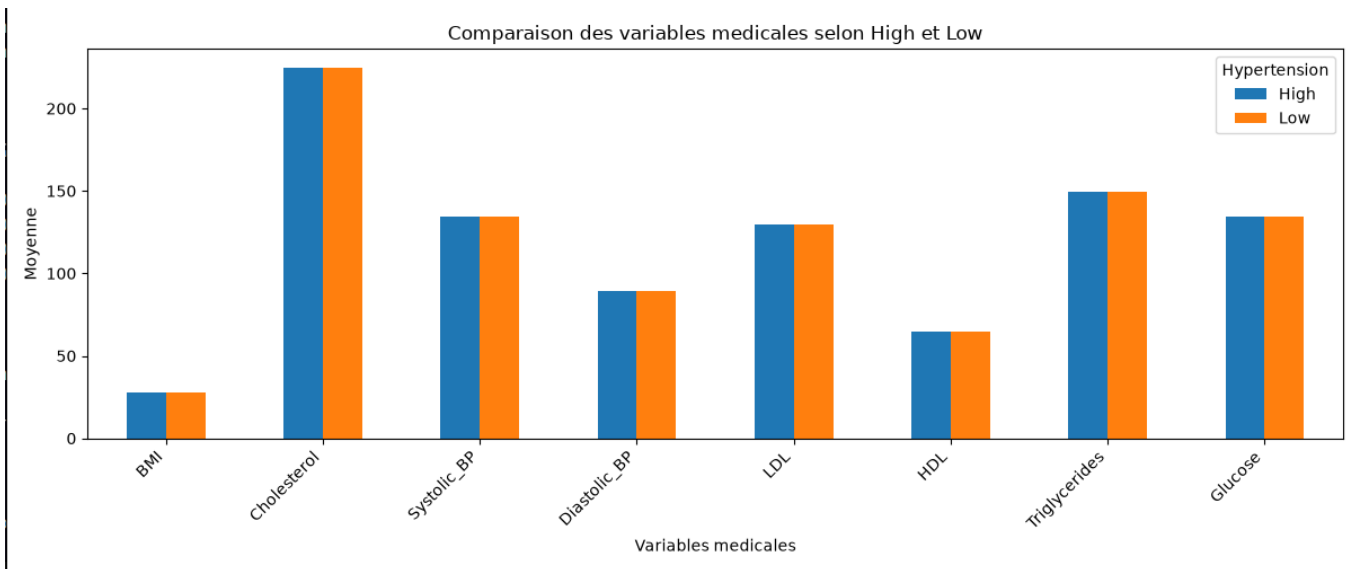
Limites observées dans le dataset

Une première limite importante du dataset est le déséquilibre des classes. La classe **High** est majoritaire et représente environ 71,88 % des données, tandis que la classe **Low** représente environ 28,12 %. Ce déséquilibre influence fortement les modèles, car un modèle peut obtenir une accuracy relativement élevée simplement en prédisant presque toujours la classe majoritaire **High**. Cela explique pourquoi l'accuracy seule n'est pas suffisante pour évaluer les performances du modèle.



Une deuxième limite apparaît lors de l'analyse des variables médicales. Le graphique comparant les moyennes des variables médicales selon les classes **High** et **Low** montre que les valeurs moyennes sont très proches pour les deux classes. Par exemple, les moyennes du BMI, du cholestérol, de la pression artérielle systolique et diastolique, du LDL, du HDL, des triglycérides et du glucose sont presque similaires entre les individus classés **High** et ceux classés **Low**.

Cette faible différence entre les deux groupes rend la séparation des classes plus difficile pour les modèles de Machine Learning. En effet, si les variables explicatives ne présentent pas de différences marquées entre les classes, l'arbre de décision a plus de difficulté à trouver des seuils pertinents permettant de distinguer clairement les individus **High** des individus **Low**.



Ainsi, les performances limitées des modèles peuvent s'expliquer par deux facteurs principaux : le déséquilibre des classes et la faible séparation des variables médicales entre les groupes High et Low.

En conclusion, cette implémentation manuelle a permis de mieux comprendre le fonctionnement interne d'un arbre de décision, notamment le choix des seuils, le calcul de l'impureté et la construction récursive. Les performances restent limitées, principalement à cause du déséquilibre des classes et de la faible séparation entre les variables médicales des classes High et Low. Cependant, ce modèle constitue une base pédagogique utile avant l'utilisation de modèles plus optimisés comme ceux de scikit-learn ou les méthodes d'ensemble.

Arbre de décision avec scikit-learn

Après avoir implémenté un arbre de décision manuellement, j'ai utilisé l'implémentation proposée par la bibliothèque scikit-learn à travers la classe `DecisionTreeClassifier`. L'objectif était de comparer mon arbre codé à la main avec une version optimisée, plus complète et plus performante, tout en gardant la même logique générale : séparer progressivement les données en fonction de conditions simples afin de prédire la classe `Hypertension`.

Comme pour les autres modèles, la variable cible `Hypertension` a été encodée en deux classes : `Low` est représentée par `0` et `High` par `1`. Les données ont ensuite été séparées en trois ensembles : un ensemble d'entraînement, un ensemble de validation et un ensemble de test. L'ensemble d'entraînement sert à apprendre le modèle, l'ensemble de validation sert à comparer les hyperparamètres, et l'ensemble de test est réservé pour l'évaluation finale.

Les variables numériques ont été normalisées avec `StandardScaler`, tandis que les variables catégorielles ont été transformées avec `OneHotEncoder`. Le prétraitement a été appris uniquement sur l'ensemble d'entraînement avec `fit_transform`, puis appliqué aux ensembles de validation et de test avec `transform`, afin d'éviter toute fuite de données.

Fonctionnement du modèle

Le modèle `DecisionTreeClassifier` fonctionne selon le même principe général que l'arbre codé manuellement. Il cherche à construire un arbre composé de nœuds de décision. À chaque nœud, le modèle choisit une variable et un seuil permettant de séparer les données en deux groupes. Les individus qui vérifient la condition vont dans une branche, tandis que les autres vont dans l'autre branche.

Le modèle répète cette opération jusqu'à atteindre une condition d'arrêt, par exemple une profondeur maximale, un nombre minimal d'échantillons pour continuer à diviser un nœud, ou une feuille suffisamment pure. À la fin, chaque feuille de l'arbre correspond à une prédiction, ici 0 pour Low ou 1 pour High.

Contrairement à mon implémentation manuelle, l'arbre de scikit-learn est optimisé. Il teste les séparations de manière plus efficace et propose davantage d'hyperparamètres pour contrôler la complexité de l'arbre, gérer le déséquilibre des classes et limiter le surapprentissage.

Hyperparamètres testés

Plusieurs hyperparamètres ont été testés afin d'observer leur influence sur les performances du modèle. Les principaux paramètres étudiés sont les suivants :

- **criterion** : définit le critère utilisé par l'arbre pour mesurer la qualité d'une séparation. J'ai testé notamment **gini** et **entropy**. Ces deux critères ont un objectif similaire : mesurer à quel point les classes sont mélangées dans un nœud afin de choisir la meilleure séparation. La différence principale vient de leur formule de calcul. Le critère **gini** mesure l'impureté d'un groupe, tandis que **entropy** mesure l'incertitude liée à la répartition des classes. Dans les deux cas, l'arbre cherche à réduire cette valeur afin d'obtenir des groupes les plus purs possible.
- **max_depth** : limite la profondeur maximale de l'arbre. Une profondeur faible produit un arbre simple, mais qui peut être insuffisant pour apprendre les différences entre les classes. Une profondeur élevée permet d'apprendre des règles plus détaillées, mais augmente le risque de surapprentissage.
- **min_samples_split** : définit le nombre minimal d'échantillons nécessaires pour diviser un nœud. Une valeur faible permet plus de séparations, tandis qu'une valeur élevée limite la complexité de l'arbre.
- **min_samples_leaf** : impose un nombre minimal d'échantillons dans chaque feuille. Ce paramètre permet d'éviter des feuilles contenant trop peu d'individus.
- **max_features** : limite le nombre de variables considérées à chaque séparation. L'utilisation de **sqrt** permet de ne tester qu'une partie des variables à chaque nœud, ce qui peut réduire le surapprentissage.
- **max_leaf_nodes** : limite le nombre maximal de feuilles dans l'arbre. Cela permet de contrôler directement la complexité du modèle.
- **min_impurity_decrease** : impose une amélioration minimale de l'impureté pour accepter une séparation. Une valeur plus élevée rend l'arbre plus conservateur.
- **ccp_alpha** : correspond à un paramètre d'élagage. Il permet de simplifier l'arbre après sa construction afin de limiter le surapprentissage.

- `class_weight` : permet de gérer le déséquilibre des classes. Avec `class_weight="balanced"`, le modèle donne plus de poids à la classe minoritaire `Low`, afin de ne pas prédire uniquement la classe majoritaire `High`.

Le dataset étant déséquilibré, le paramètre `class_weight="balanced"` a été particulièrement important. Sans prise en compte du déséquilibre, le modèle avait tendance à favoriser la classe majoritaire `High`, ce qui augmentait l'accuracy mais diminuait fortement la capacité du modèle à reconnaître la classe `Low`.

Influence des hyperparamètres sur les performances

Les tests réalisés ont montré que les hyperparamètres influencent fortement le comportement de l'arbre.

Lorsque l'arbre est trop simple, par exemple avec une profondeur faible ou des contraintes trop fortes, il prédit très souvent la classe majoritaire `High`. Dans ce cas, l'accuracy peut sembler élevée, car la classe `High` est majoritaire dans le dataset. Cependant, la matrice de confusion montre que la classe `Low` est très mal reconnue.

À l'inverse, lorsque l'arbre est plus profond et moins contraint, il devient capable de reconnaître davantage d'individus `Low`. Cependant, cette amélioration se fait souvent au prix d'une baisse de l'accuracy globale et d'une augmentation du nombre de vrais `High` prédits à tort comme `Low`. Dans un contexte médical, cette erreur est importante, car elle revient à classer une personne réellement à risque comme non à risque.

L'utilisation de `class_weight="balanced"` permet de réduire le biais vers la classe majoritaire. Le modèle accorde alors plus d'importance à la classe `Low`, ce qui améliore son rappel. Cependant, ce choix peut aussi diminuer l'accuracy globale, car le modèle prédit moins systématiquement la classe `High`.

Le paramètre `max_features="sqrt"` a également été retenu. Il permet de ne considérer qu'une partie des variables à chaque séparation. Cela peut rendre l'arbre moins dépendant de certaines variables et réduire le risque de surapprentissage. Dans mes essais, cette option a permis d'obtenir un compromis plus intéressant entre la détection de la classe `High` et la reconnaissance partielle de la classe `Low`.

Choix des paramètres retenus

Après plusieurs essais, la configuration retenue pour l'arbre de décision scikit-learn est la suivante :

```
DecisionTreeClassifier(  
    criterion="gini",  
    splitter="best",  
    max_features="sqrt",  
    class_weight="balanced",  
    max_depth=46,  
    min_samples_split=2,  
    random_state=1  
)
```

Le critère `gini` a été conservé car il donnait un meilleur compromis dans mes tests, notamment pour la reconnaissance de la classe `Low`. Les critères `entropy` et `log_loss` donnaient parfois une accuracy légèrement plus élevée, mais ils détectaient moins bien la classe minoritaire.

Le paramètre `splitter="best"` signifie que le modèle choisit à chaque nœud la meilleure séparation possible selon le critère choisi. Ce choix est cohérent avec l'objectif d'obtenir les séparations les plus efficaces.

Le paramètre `max_features="sqrt"` a été retenu afin de limiter le nombre de variables testées à chaque séparation. Cela permet de contrôler la complexité du modèle et de réduire le risque de dépendre toujours des mêmes variables.

Le paramètre `class_weight="balanced"` a été choisi à cause du déséquilibre du dataset. La classe **High** est majoritaire, ce qui peut pousser le modèle à prédire trop souvent cette classe. En utilisant `balanced`, le modèle accorde plus d'importance à la classe **Low**, ce qui permet d'améliorer sa détection.

Le paramètre `max_depth=46` permet à l'arbre d'apprendre des règles suffisamment détaillées. Une profondeur trop faible donnait un modèle trop simple, tandis qu'une profondeur plus élevée n'apportait pas forcément une amélioration significative. Cette valeur a donc été retenue après plusieurs tests comme un compromis entre complexité et performance.

Enfin, `min_samples_split=2` autorise le modèle à continuer les séparations tant qu'un nœud contient au moins deux individus. Ce choix rend l'arbre plus flexible et lui permet de construire des règles plus détaillées. Comme cette flexibilité peut augmenter le risque de surapprentissage, elle est compensée en partie par l'utilisation de `max_features="sqrt"` et par l'évaluation sur l'ensemble de validation.

Résultats obtenus

Avec cette configuration, les résultats obtenus sur l'ensemble de validation sont les suivants :

```
Accuracy : 0.5978969025031432
Precision High : 0.7196997727152598
Recall High : 0.7216833624847617
F1 High : 0.7206902027205844
Precision Low : 0.2834334061135371
Recall Low : 0.28143631436314365
F1 Low : 0.28243132988849606
```

La matrice de confusion obtenue est :

```
[[ 2077  5303]
 [ 5251 13616]]
```

Cette matrice montre que le modèle reconnaît correctement 13 616 individus de la classe **High** et 2 077 individus de la classe **Low**. Cependant, 5 251 vrais **High** sont prédits à tort comme **Low**, et 5 303 vrais **Low** sont prédits à tort comme **High**.

Ces résultats montrent que le modèle détecte mieux la classe **Low** que les configurations qui maximisent uniquement l'accuracy. En revanche, cette amélioration s'accompagne d'une diminution de l'accuracy globale et d'une augmentation des erreurs sur la classe **High**.

Interprétation

Le choix du modèle final ne peut donc pas se faire uniquement à partir de l'accuracy. Dans ce dataset, la classe **High** est majoritaire. Un modèle qui prédit très souvent **High** obtient facilement une accuracy élevée, mais il reconnaît mal la classe **Low**.

L'arbre de décision scikit-learn retenu cherche donc un compromis : il ne maximise pas seulement l'accuracy, mais il essaie aussi de mieux prendre en compte la classe minoritaire **Low**. Cela explique pourquoi l'accuracy est plus faible qu'un modèle très orienté vers **High**, mais pourquoi le rappel de la classe **Low** est meilleur.

Cependant, dans un contexte médical, les vrais **High** prédits à tort comme **Low** restent une erreur importante. Le modèle retenu doit donc être interprété avec prudence. Il permet d'améliorer la reconnaissance de la classe minoritaire, mais il ne doit pas être considéré comme un outil de diagnostic. Il s'agit d'un modèle prédictif indicatif, utile pour comparer différentes approches de Machine Learning sur ce dataset.

Importance des variables

L'un des avantages de `DecisionTreeClassifier` est qu'il permet d'obtenir l'importance des variables avec l'attribut `feature_importances_`. Cette information indique quelles variables ont le plus contribué aux décisions de l'arbre.

Sauvegarde des résultats

Les résultats des trois modèles ont été sauvegardés dans des fichiers CSV nommés `resultat_majority_baseline.csv`, `resultat_arbre_by_me.csv` et `resultat_arbre_sklearn.csv`. Ces fichiers contiennent les principales métriques des modèles : accuracy, précision, rappel et F1-score pour les classes **High** et **Low**.

Cette sauvegarde permet ensuite de regrouper les résultats dans un tableau comparatif afin d'analyser plus facilement les performances des différents modèles.